

MODULE 9 · TROUBLESHOOTING

# Ingress Problems

503, 404, TLS, or wrong class — the status names the layer

---

# An Ingress request crosses five layers before it succeeds

A request passes through the **ingress controller**, an **Ingress rule**, a **Service**, that Service's **EndpointSlice**, and — for HTTPS — a **TLS Secret**. Break any one layer and you get a different, very specific symptom.

Four failures cover almost everything you'll see: **503** (backend), **404** (routing), a missing or wrong **TLS Secret**, and an **ingressClassName** mismatch that gets the whole Ingress ignored.

## BY THE END YOU CAN

- Read a status code back to its layer
- Tell a default-backend 404 from an app 404
- Spot an Ingress no controller is watching

## THE TRIAGE FLOW

# Walk the path top to bottom



Each stage fails a different way and needs a different command. Don't read controller logs until you've confirmed the Ingress object itself is correct.

# What the response is telling you

| STATUS                                    | COMMON CAUSE                                                            | GO-TO COMMAND                                                                     |
|-------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 503                                       | Backend Service missing, or has no ready endpoints                      | <code>kubectl get endpointslices -l kubernetes.io/service-name=&lt;SVC&gt;</code> |
| 404 (default backend)                     | No host/path rule matched the request                                   | <code>describe ingress</code> — check rules                                       |
| 404 (from your app)                       | Rule matched; the app itself has no route there                         | check the app, not the Ingress                                                    |
| TLS "fake certificate"                    | Secret in <code>tls:</code> block missing or misnamed                   | <code>kubectl get secret &lt;name&gt;</code>                                      |
| No <code>ADDRESS</code> , nothing happens | <code>ingressClassName</code> missing/wrong — no controller is watching | <code>kubectl get ingressclass</code>                                             |

# Three commands, in this order

- **describe first** — rules, backend, and the TLS block, all in one place
- **controller logs second** — did nginx see this Ingress, and what did it do
- **curl -v third** — the actual status code, and for TLS the cert subject

## ORDER MATTERS

A wrong `ingressClassName` means the controller logs won't mention your Ingress at all — that silence is itself the diagnosis.

```
$ kubectl describe ingress NAME
# Rules, Backends, TLS – one place

$ kubectl logs -n ingress-nginx \
  -l app.kubernetes.io/name=ingress-nginx --tail=20

$ curl -v http://NODE_IP:PORT/PATH
```

## The Ingress object can look perfectly valid

describe shows rules and a TLS block that all read fine — but the controller can only forward where the Service points. No Service, or a Service with no ready pod behind it, both return **503**.

Fix = correct `backend.service.name` to a Service that exists and has endpoints; 503 becomes 200 on the next request.

```
$ kubectl get svc -n training
# no webapp-typo-svc in the list

$ kubectl logs -n ingress-nginx ...
# service "training/webapp-typo-svc"
# does not exist
```

## Two 404s can look identical

A **default-backend 404** (no rule matched) and an **app-side 404** (rule matched, app has no route) return the same status code and near-identical HTML. Don't stop at the status code.

The tell is in the body's footer — a bare `nginx` (version scrubbed) is ingress-nginx's own default backend; a versioned string means the upstream pod actually answered. The access log confirms it from the other side: a default-backend 404 has no upstream column populated at all.

| REQUEST | RESPONDER       | FOOTER TELL  |
|---------|-----------------|--------------|
| /       | default backend | nginx        |
| /api    | webapp pod      | nginx/1.25.x |

**NOTE**

Modern ingress-nginx also strips the `Server:` header — the body is the reliable tell, not the headers.

# Where people trip

- **Trusting a matching Ingress YAML.** The object can be perfect and the Service still 503.
- **Stopping at "404."** Same code, two very different causes.
- **Forgetting ingressClassName.** The Ingress sits idle with no ADDRESS, no logs, no complaint.
- **Assuming HTTPS works means the Secret exists.** The "fake certificate" fallback is silent, not an error.



# Commands to keep at hand

```
$ kubectl describe ingress NAME -n NS          # rules + backend + TLS, one place
$ kubectl get ingressclass                      # confirm the class exists
$ kubectl get endpointslices -n NS -l kubernetes.io/service-name=SVC # backend has ready pods?
$ kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=20
$ curl -v http://NODE_IP:PORT/PATH             # actual status + cert
```

**Tip:** the status code narrows the layer; describe + controller logs name the cause.

## RECAP

# Ingress triage in one breath

- Request crosses controller → rule → Service → EndpointSlice → TLS, in order
- 503 = backend; 404 = routing (two different responders)
- TLS "fake certificate" = missing or wrong Secret
- ingressClassName mismatch = Ingress silently ignored

### HANDS-ON

Now run Lab 9.6  
[labs/9.6/](#)

Next: [9.7 — PVC Problems](#)